IPAP
RÍO NEGRO

TECNICATURA SUPERIOR EN DESARROLLO DE SOFTWARE FULL STACK

MATERIA: PROGRAMACIÓN FRONTEND

PROFESOR: Lisandro Ojeda Fernández

2° Cuatrimestre de 2026

ACTIVIDAD FINAL INTEGRADORA
Segunda Entrega

Responsables:

- Boock, Ayelen. DNI 29165794.
- Chavez Neicul, Verónica. DNI 18856116.

Fecha de entrega: 02 de noviembre de 2026.

CONSIGNAS:

1. Presentación de la actividad

Cada estudiante o equipo deberá desarrollar una aplicación frontend, aplicando los contenidos trabajados en la asignatura. El proyecto deberá ser sencillo, funcional y comprensible. No se exigirá programación avanzada: se valorará que el estudiante pueda justificar sus decisiones, explicar el código utilizado y demostrar que comprende su funcionamiento.

Tecnologías y alcance

• HTML para organizar el contenido y comunicar la función de cada región.

• CSS para la presentación visual, el modelo de caja y la adaptación a diferentes pantallas.

• JavaScript para la interactividad, los eventos, las validaciones y la manipulación del DOM.

• Variables, arreglos, objetos, JSON y, cuando corresponda, localStorage para representar y conservar datos.

• PHP podrá incorporarse de manera opcional, pero no será un requisito obligatorio.

• Se podrán utilizar librerías sencillas siempre que el estudiante pueda explicar su propósito.

Fuera del alcance: No será necesario implementar bases de datos, autenticación real, pagos electrónicos, servidores complejos ni algoritmos avanzados.

2. Segunda y última entrega - 02/11

Segundo parcial: aplicación terminada y explicación final del código

La aplicación deberá estar terminada, probada y preparada para su demostración. También deberán incorporarse las correcciones indicadas durante la primera entrega.

Requisitos generales

• Interfaz completa, ordenada, legible y visualmente coherente.

• Navegación clara y diseño adaptable a diferentes tamaños de pantalla.

• Formularios con etiquetas, validaciones y mensajes comprensibles.

• Manipulación del DOM para mostrar, agregar, modificar o eliminar información.

• Uso de eventos como click, submit, input o change.

• Utilización de variables, funciones, arreglos y objetos.

• Representación de información mediante JSON.

• Persistencia con localStorage, cuando resulte pertinente.

• Código organizado en archivos HTML, CSS y JavaScript.

• Pruebas, depuración de errores y explicación final de las funciones principales.

Funcionalidades orientativas por proyecto

Proyecto	Funciones mínimas
Calculadora de gastos personales	Registrar concepto, categoría y monto; validar los datos; mostrar los gastos en una lista; calcular el total; eliminar registros; conservar la información localmente.
Encuesta de satisfacción	Ingresar nombre; seleccionar una valoración; escribir un comentario; validar campos obligatorios; mostrar un resumen; guardar y listar las respuestas.
Galería de imágenes interactiva	Mostrar imágenes; organizar por categorías; filtrar la galería; ampliar una imagen seleccionada; avanzar o retroceder entre imágenes; cerrar la vista ampliada.
Sistema de turnos	Seleccionar datos; validar; registrar; listar; cancelar o modificar; conservar información localmente.
Tienda virtual	Mostrar catálogo; buscar o filtrar; agregar al carrito; cambiar cantidades; eliminar; calcular el total.
Panel académico	Registrar estudiantes, materias y notas; mostrar tablas; buscar; editar; eliminar; calcular promedios.

DESARROLLO:

DESCRIPCIÓN DEL PROYECTO:

En el presente proyecto, se presenta un panel de gestión académica para el Instituto San Agustín, una institución ficticia. Esta aplicación web cuenta con dos tipos de acceso: administración y profesor. La administración da de alta y gestiona carreras, profesores, materias y estudiantes, y permite consultar sus calificaciones. El profesor carga notas y asistencias, y consulta el listado de estudiantes con sus promedios.
Todos los datos se guardan en el navegador con localStorage, sin servidor ni base de datos, como lo solicita la consigna.

AVANCES RESPECTO DE LA PRIMERA ENTREGA:

En la primera entrega, el desarrollo del proyecto constaba de la estructura HTML, los estilos (css) y la validación del login. Para esta segunda entrega, había quedado como pendientes:

- Almacenamiento de datos en localStorage: se implementó en el alta y registro de: carreras, profesores, materias, estudiantes, notas y asistencias.
- Validación de DNI: se implementó una validación para que solo se puedan ingresar números y al menos 7 caracteres en estudiantes, profesores y recuperación de usuario/contraseña. También se aplicó el mismo criterio para la validación del CUIL en profesores.
- Código de materia con formato MAT-123: se implementó en el alta de materias, con control para que no se puedan repetir los códigos en materias diferentes. También se implementó la misma validación para el alta de carreras, con código CAR-123, sumando a esta además que cuando se quiera editar, no se pueda cambiar el código ingresado al dar el alta.
- Notas dentro del rango 1-10: en la carga de notas, sólo se pueden ingresar números de 1 a 10, enviando un mensaje de error si se intentan cargar fuera del rango.
- Cálculo de promedio: se realiza por estudiante y promedio general por materia. Tanto las notas como el promedio se guardan en localStorage para su consulta posterior.
- Fecha no futura: se implementó en la carga de asistencias. Esta permite la carga de asistencias en la fecha del día y en anteriores. También se almacenan en localStorage.

- Cierre de sesión: simulado, el enlace vuelve a la página de inicio, no se implementó un login real.

FUNCIONES IMPLEMENTADAS EN EL PROYECTO:

Inicio y acceso:

- En la página de inicio: acceso según usuario (profesor y administración), cada uno envía el rol en la dirección login.html?rol=admin o ?rol=profesor, y el login lo lee con URLSearchParams para llevar a cada usuario a su panel.
- Login con validación de campos (evitar campos vacío, extensión mínima de contraseña de 4 caracteres).
- Recuperación de usuario y contraseña con validación del número de documento.

Panel de administración:

- Carreras: alta, edición, eliminación y buscador por nombre de materia. En la edición de carreras, se impide que el código de la misma se edite, quedando de solo lectura, y además no se puede eliminar una carrera si tiene materias asignadas.
- Profesores: alta, edición, eliminación y buscador por apellido. La tabla de registro muestra las materias que dicta cada profesor, y no se permite eliminar a quienes tengan materias asignadas.
- Materias: alta con carrera, profesor a cargo, año, duración (anual o cuatrimestral), año lectivo y elección de días y horarios de cursada (de 1 a 4 días). La tabla de registro muestra además el nombre de la carrera y del profesor, permite editar, eliminar y buscar materias por nombre.
- Estudiantes: alta, edición, eliminación y buscador por apellido.
- Gestión académica: tablas de estudiantes, materias y notas, con buscador, edición y eliminación.
- Consulta de calificaciones: en el panel académico, permite consultar las notas y promedio de cada estudiante por materia, filtrar por apellido y por materia, y visualizar el promedio general de la materia elegida.

En estudiantes y profesores se implementó la validación del DNI para que tenga al menos siete caracteres numéricos.

Panel del profesor:

- Carga de notas: se elige la materia y se cargan hasta tres notas por estudiante. El promedio se actualiza mientras se van cargando las notas. Al guardar, si el estudiante ya tenía notas registradas, se actualizan y no se duplican.

- Carga de asistencias: se elige la materia y la fecha (aparece por defecto la fecha del día), y se marca Presente o Ausente para cada estudiante. Si en esa fecha ya se había cargado la asistencia, aparece marcada.
- Listado de estudiantes: misma consulta de notas y promedios, con filtros por apellido y por materia.

ORGANIZACIÓN DE LOS ARCHIVOS:

Los archivos están separados por tipo en diferentes carpetas. Creamos una carpeta html/ para los archivos con esta extensión, los archivos JavaScript relacionados los agrupamos en la carpeta js/ y la hoja de estilos en css/. El index.html lo dejamos en la raíz del proyecto porque es el nombre que los servidores web abren por defecto cuando alguien entra a una carpeta sin pedir un archivo puntual. Si estuviera dentro de html/, en la raíz no habría nada para mostrar, y el servidor daría un error o mostraría la lista de archivos.

Cada archivo .html está relacionado con un archivo JavaScript que maneja las funciones de la página:

- index.html trabaja con alert_usuario.js, que lanza un aviso si alguien quiere registrarse.
- login.html utiliza validacion_login.js para validar usuario y contraseña y redireccionar según el rol elegido.
- recuperar_usuario_contrasena.html utiliza alert_recuperacion.js para validar el documento y dar el mensaje de recuperación.
- alta_carreras.html y registro_carreras.html utilizan validacion_alta_carreras.js y registro_carreras para dar de alta, editar, dibujar la tabla, buscar y eliminar.
- alta_profesor.html y registro_profesores.html se relacionan con validacion_alta_profesores.js y registro_profesores.js para el alta, la edición, el dibujo de la tabla, buscar y eliminar.
- alta_materias.html y registro_materias.html se relacionan con validacion_alta_materias.js y registro_materias.js para el alta, la edición, el dibujo de la tabla, buscar y eliminar.
- alta_estudiantes.html y registro_estudiantes.html se relacionan con validacion_alta_estudiantes.js y registro_estudiantes.js para el alta, la edición, el dibujo de la tabla, buscar y eliminar.
- carga_notas.html utiliza carga_notas.js para la carga de las notas, la validación del rango de notas entre 1 y 10 y el cálculo del promedio.
- carga_asistencias.html se relaciona con carga_asistencias para el registro de la

asistencia, la validación de fecha no futura.

- listado_estudiantes.html y consultar_notas.html se relacionan con un solo archivo listado_estudiantes.js para mostrar las notas por estudiante y por materia, los promedios y filtros aplicados.

CONCEPTOS DE JAVASCRIPT UTILIZADOS:

1. Selección del DOM: el DOM es la representación del página que JavaScript puede leer y modificar. En el proyecto utilizamos *getElementById("id")* para buscar un elemento por su id, por ejemplo:

```
document.getElementById("materia");
```

También utilizamos *querySelector("selector")*, que busca el primer elemento que cumple un selector de CSS, y *querySelectorAll("selector")*, que busca todos los que cumplen el selector y los devuelve en una lista, como por ejemplo el radio marcado de una fila:

```
querySelectorAll('input[name="dia"]:checked');
```

2. Eventos: un evento es algo que pasa en la página. Con addEventListener se avisa a JavaScript qué función ejecutar cuando pasa:

- submit: cuando se envía un formulario, por ejemplo, las altas que se validan y guardan al tocar "Guardar".
- click: cuando se hace clic, por ejemplo, en el botón guardar asistencia.
- input: cuando se escribe en un campo, en cada tecla, por ejemplo, cuando se cargan las notas el promedio se actualiza mientras se escribe.
- change: cuando se cambia un select o una fecha: al elegir otra materia, la tabla se vuelve a dibujar.
- preventDefault(): por defecto, un formulario que se envía recarga la página, event.preventDefault() impide que se recargue la página, así JavaScript puede validar y guardar sin que se pierda lo escrito.

3. Creación de elementos: las tablas no están escritas en el HTML, sino que JavaScript las arma con los datos guardados:

- createElement(tr) crea un elemento nuevo, que todavía no se ve.
- textContent le pone texto, es más seguro que innerHTML porque no interpreta etiquetas HTML.
- appendChild(elemento) lo agrega dentro de otro elemento, y recién ahí aparece en la página.
- colSpan hace que una celda ocupe varias columnas.

4. Métodos de arreglos:

- forEach: para recorrer los arreglos y ejecutar algo con cada elemento.

- for…of: también recorre, pero permite cortar con return o saltear con continue. Al guardar asistencias, por ejemplo, corta si falta marcar a alguien.
- push: agrega un elemento al final, por ejemplo, al guardar un estudiante o profesor nuevos.
- splice: saca elementos de una posición, por ejemplo, al eliminar un registro.
- find: Devuelve el primer elemento que cumple una condición, como cuando se busca la carrera de una materia por su código.
- findIndex: devuelve la posición del elemento, o -1 si no está. La utilizamos para saber si ya había notas para actualizarlas en lugar de duplicarlas.
- filter: devuelve un arreglo nuevo con los elementos que cumplen la condición, por ejemplo, las materias de un profesor.
- some: devuelve true si alguno cumple la condición, como no borrar un profesor si tiene materias asignadas.
- map: transforma cada elemento y devuelve un arreglo nuevo, por ejemplo, para pasar de objetos de materias a solo sus nombres.
- join: une los elementos en un texto con separador, por ejemplo, cuando en la tabla se muestran los horarios.

5. Datos: cada tipo de datos es un arreglo de objetos. Se guardan el localStorage convertido a texto con JSON.stringify y se recupera con JSON.parse. Si la clave todavía no existe, se usa un arreglo vacío (|| [ ]).

Clave	Campos de cada objeto
carreras	codigo, nombre, duracion, tituloOtorga, plan
profesores	apellido, nombre, dni, cuil, fechaNac, lugarNac, direccion, telefono, titulo, institucion
materias	codigo, asignatura, anio, **carreraCodigo**, horarios [{dia, horario}], **profesorCuil**, duracion, lectivo
estudiantes	apellido, nombre, dni, fechaNac, lugarNac, direccion, telefono, urgencia
notas	**dni**, **materiaCodigo**, notas [nota1, nota2, nota3] (las vacías se guardan como null)
asistencias	**dni**, **materiaCodigo**, fecha, estado ("P" o "A")

Para no repetir información, los objetos guardan solo el dato que identica a otro objeto (los marcados en negrita), y el nombre se busca con find al mostrar la tabla, por ejemplo:

- materia.carreraCodigo guarda el código de la carrera.
- materia.profesorCuil guarda el CUIL del profesor a cargo.
- En notas y asistencias, el dni identifica al estudiante y materiaCodigo a la materia. En asistencas se suma la fecha, para guardar un registro por día.

VALIDACIONES:

- Login: usuario y contraseña obligatorios, contraseña de al menos 4 caracteres.
- Recuperación: número de documento obligatorio y solo con números.
- Alta de carrera: campos obligatorios, código con formato CAR-123, plan de cuatro números, código no repetido.
- Alta de profesor: campos obligatorios, DNI numérico de al menos 7 caracteres, CUIL numérico de al menos 10.
- Alta de materia: campos obligatorios, código con formato MAT-123, año lectivo de 4 números, entre 1 y 4 días, cada uno con su horario, código no repetido.
- Alta de estudiante: campos obligatorios, DNI numérico de al menos 7 caracteres.
- Carga de notas: materia elegida, notas entre 1 y 10.
- Carga de asistencias: materia y fecha elegidas, fecha no futura, todos los estudiantes marcados.

Los mensajes de error se muestran en un párrafo debajo del formulario, con textContent, para que el usuario sepa qué corregir sin perder lo que ya escribió.

LÍMITES DEL ALCANCE:

- El login es simulado, valida los campos y lleva a cada rol a su panel, pero no comprueba usuarios reales.
- No hay inscripción a materias, por esto, en la carga de notas y asistencias aparecen todos los estudiantes en todas las materias.
- El profesor ve todas las materias, ya que el login no guarda qué profesor ingresó.
- Los datos quedan en el navegador, localStorage guarda la información en cada navegador por separado, si se abre el proyecto en otra computadora, empieza vacío.
- No se desarrollaron las páginas de perfil, mensajes, mesas de examen y cronograma de horarios.

TECNOLOGÍAS UTILIZADAS:

- HTML 5

- CSS3 para el diseño, el modelo de caja y la adaptación a pantallas pequeñas.
- JavaScript para la interactividad, eventos, validaciones y manipulación del DOM.
- JSON y localStorage para representar y guardar los datos necesarios.
- Git y GitHub para el control de versiones y la entrega.
- Canva para los bocetos previos.
- Extensión Live Preview.
- Claude Code (Anthropic)

USO DE INTELIGENCIA ARTIFICIAL

Para el desarrollo del trabajo se utilizó la aplicación Claude Code (Anthropic) como asistente en las siguientes tareas:

- Revisión y orientación: en la revisión del proyecto para detectar errores y requisitos faltantes, explicación de conceptos como localStorage, eventos, métodos de arreglos, URLSearchParams, selectores; detección de errores de ids, nombres de propiedades y rutas. También orientó en cómo relacionar las materias, carreras y profesores, la carga de notas con el cálculo de promedios y la consulta de calificaciones.
- Código generado por la IA: parte de las validaciones de los formularios de alta; las tablas de registro con los botones de editar y eliminar, modo edición y los buscadores.

Todo el código fue revisado, adaptado y probado por el equipo, y se comprende su funcionamiento.